

ICU-X0201 HIGH FRAME RATE EXAMPLE USER GUIDE

INTRODUCTION

This example program demonstrates how to build and run an ultrasonic sensing application using the Chirp SonicLib API and driver for ICU-x0201 sensors. The application runs on the SmartSonic2 or Nucleo board, and it can operate with either a single sensor or multiple sensors connected to the board. This application showcases the high-frame-rate mode of the ICU-x0201 sensors, which uses a double-buffering scheme within the sensor to maximize the device frame-rate.

REQUIRED EQUIPMENT

- TDK SmartSonic2 board or ST Nucleo F446ZE board
- TDK Ultrasonic ToF EVK board
- ICU-x0201 sensor module(s) with flex cable(s)
- Micro-USB cable(s)

REQUIRED SOFTWARE PACKAGES

- **Invn.chirpmicro.BOARD.icux0201-highframerate-example.X_X_X.zip** (actual file name will include version number), includes:
 - BOARD is one of smartsonic2 or nucleo
 - “Hello Chirp” application source files
 - Chirp SonicLib sensor API and driver files. The application requires SonicLib v4 or later.
 - Sensor firmware image files
 - Board support package files for SmartSonic2 or nucleo board
 - MPLAB X project files to build the application (smartsonic2)
 - STM32Cube IDE project files to build the application (nucleo)
- [MPLAB X](#) integrated development environment – download from Microchip.com or STM32 Cube IDE
- Terminal emulator of your choice (for example TeraTerm or PuTTY)

TABLE OF CONTENTS

INTRODUCTION	2
REQUIRED SOFTWARE PACKAGES	2
1 INSTALLATION / PREPARATION	4
2 BUILDING AND PROGRAMMING THE EXAMPLE APPLICATION AND SONICLIB	5
3 RUNNING THE EXAMPLE APPLICATION	6
4 SENSOR CONFIGURATION SETTINGS	7
4.1 SETTINGS AVAILABLE IN APP_CONFIG.H	7
4.2 CHANGING THE MEASUREMENT INTERVAL	7
4.3 EDITING THE MEASUREMENT CONFIGURATION	8
4.4 READING DATA	18
4.5 SPI DMA INTERFACE	20
5 THEORY OF OPERATION	21
6 REVISION HISTORY	23

LIST OF FIGURES

Figure 1. Data valid timing diagram	19
Figure 2. Logic analyzer capture of double-buffer mode	21
Figure 3. Metadata embedded in IQ data	22

LIST OF TABLES

Table 1. Frequency range of ICU series parts	8
Table 2. Round robin transmit example	10
Table 3. IQ buffer when metadata option is enabled	11
Table 4. PMUT Command Structure	12
Table 5. PMUT Commands	12
Table 6: Count instruction format	12
Table 7: TX instruction format	12
Table 8: RX instruction format	12
Table 9: EOF instruction format	13
Table 10. PMUT Command Register Descriptions	13
Table 11. Measurement queue memory map	15
Table 12. Individual measurement config map (meas[0] and meas[1])	15
Table 13. Instruction extension registers	16
Table 14. Additional measurement configuration registers	16
Table 15. Custom sequence registers	17
Table 16. cur_meas to config mapping	17
Table 17. IQ data relative address	18
Table 18. SPI DMA Command Format	20
Table 19. ICU PMEM and DMEM Access	20

1 INSTALLATION / PREPARATION

- Optionally, download and install the **ICU-x0201 Evaluation Kit EVK** Application. The EVK is not required to use this example, but you can use the tool to experiment with different measurement configurations, then export the settings to use with Hello Chirp.
- Install terminal emulator.
- ***Note:** This example application only requires one cable, because the Debug Port is used for both the IDE connection and the serial output from the program. Other applications, notably the ICU-x0201 Evaluation Kit (EVK), may also require a second cable connected to the Applications Port, as shown in Figure 1.*

2 BUILDING AND PROGRAMMING THE EXAMPLE APPLICATION AND SONICLIB

- The example project source files are organized in three sub-directories under **source**:
 - **application** – contains **src** and **inc** directories with the Hello Chirp application
 - The **application/icux0201-hellochirp-example/src/main.c** file contains the entry point for the application along with various routines that demonstrate how to read and manage the Chirp sensor(s). See the comments in that file for detailed information about the operation of the application.
 - The **application/icux0201-hellochirp-example/inc/app_config.h** file contains various definitions that control the behavior of the application and ultrasonic sensors. These include sensor configuration values such as sensor refresh sample rate and maximum range. **Modify the settings in this file and rebuild to change the configuration.**
 - **drivers/invn-soniclib/invn/soniclib** – contains the SonicLib API and driver files, sensor firmware modules, and other distribution files from InvenSense.
 - The directory contains header files that must be included when building applications with SonicLib. In particular, the **soniclib.h** file contains the interface definitions for the SonicLib API.
 - The **drivers/invn-soniclib/invn/soniclib** directory must be in the include path when building. This is already set in the supplied project files.
 - **board** – contains support files for the SmartSonic2 and Nucleo boards.
 - The required board support package interface routines, as defined in **drivers/invn-soniclib/invn/soniclib/chirp_bsp.h**, are in the **board/smartsonic2/HAL/src/chbsp_chirp_samg55.c** file for the SmartSonic2 and in the **board/nucleo/HAL/src/chbsp_nucleo144.c** file for the Nucleo.

3 RUNNING THE EXAMPLE APPLICATION

- Start the terminal emulator program and open/configure the COM port assigned to the SmartSonic2 (or Nucleo) board (as found earlier using Windows Device Manager):
 - **921600 baud**
 - **8 bits data, no parity, 1 stop bit**
 - **New-line sequence = Line Feed only (no carriage return)**
- Reset the board using the board's reset button.
- Status messages from the application will appear on the terminal output, followed by summary data from the sensor initialization (device frequency, measurement configuration, etc.).
- Raw data from the application will be printed at a configurable interval (set in app_config.h). Due to the high frame rate, it is not possible to print every sample over the serial port.

4 SENSOR CONFIGURATION SETTINGS

The *app_config.h* header file contains symbolic definitions for parameters that affect the sensor measurements and application execution. You can change these definitions to adjust the overall operation. These parameters are documented here.

The *measurement_config.c* file contains the sensor measurement configuration(s). There are three configurations provided, which are intended to support the ICU-10201, ICU-20201, and ICU-30201. Each of these sensors runs at a different frequency, so some adjustment of the receive length is necessary to meet the desired trigger rate timing (described below).

4.1 SETTINGS AVAILABLE IN APP_CONFIG.H

This section documents the settings available in the *app_config.h* file.

4.1.1 RX_LENGTH_SMCLK

Use this define to select the measurement length.

Three different measurement length choices are provided. These are intended to be used with ICU-10201, 20201, or 30201. These parts have different nominal operating frequencies, so we need to adjust the measurement length to ensure the receive operation has time to finish before the next trigger is issued. These can be adjusted as needed if the measurement interval is changed.

Rather than using this define, you can directly edit the lengths in the *measurement_config.c* file. This is necessary if you wish to have different RX lengths for different configurations, for example.

4.1.2 USE_MINIMAL_INT_HANDLER

Enable the minimal interrupt handler in the BSP layer. This avoids the heavier interrupt handling done by *ch_interrupt()*. This is required for the high frame rate example.

4.1.3 CHIRP_IMPROVE_PITCH_CATCH_SENSITIVITY

If set, force the same operating frequency for all sensors. This improves the sensitivity in pitch-catch mode.

4.1.4 CHIRP_MEAS_OPTIMIZE

If set to 1, enable optimization of the measurement segments to improve the performance at close distances. Additional TX segments will be inserted into the sequence to implement a reverse drive operation following the main transmit.

This is NOT supported when using instruction extensions.

4.1.5 MEASUREMENT_INTERVAL_MS

This sets the overall sample rate in milliseconds. The sensor will be triggered at this rate.

4.1.6 STATUS_PRINT_INTERVAL_MS

This sets an interval in milliseconds at which to print data from the sensor. Not every frame can be printed because there is not enough bandwidth available over the UART port.

4.1.7 DATA_VALIDATION_ENABLED

When set, validate the data transfer integrity. In this mode, the raw data is replaced by a known sequence, and SonicLib will validate the sequence is received correctly.

4.2 CHANGING THE MEASUREMENT INTERVAL

The measurement interval is the time between frame triggers of the ICU-x0201. The host microcontroller will generate a trigger to the ICU-x0201 every *MEASUREMENT_INTERVAL_MS*, which is specified in *app_config.h*. This is set to 2 milliseconds by default,

which gives a 500 Hz trigger rate. This default setting with default measurement configuration works well, but you may want to adjust this for your own application. In this section we give some guidelines on how to do this.

The main requirement is that the total measurement time in milliseconds is less than `MEASUREMENT_INTERVAL_MS`. The measurement time can be found as

$$t_{meas} = (L_{tx} + L_{count} + L_{rx}) / (16f_{pmut}),$$

where L_{tx} is the total length of all transmit instructions, L_{count} is the total length of all count instructions, L_{rx} is the total length of all receive instructions, and f_{pmut} is the frequency of the PMUT. The length of the instructions is specified in the `measurement_config.c` file. This file can be generated by the ICU-x0201 EVK software or modified by hand.

We can also write an expression for the total length given a measurement time as

$$L_{total} = 16f_{pmut}t_{meas},$$

where L_{total} is the sum of the transmit, count, and receive lengths.

The PMUT operating frequency varies between ICU-10201, ICU-20201, and ICU-30201. See the table below for minimum and maximum ranges of the operating frequency.

Table 1. Frequency range of ICU series parts¹

Part	Min frequency	Max frequency
ICU-10201	173 kHz	181 kHz
ICU-20201	70 kHz	95 kHz
ICU-30201	45 kHz	60 kHz

The measurement times will be the longest at the lowest end of the frequency range, so to be conservative, you can calculate t_{meas} for the minimum frequency.

For example, if we want to receive ultrasound for 2ms, then we can calculate that L_{total} must be less than 5536 for ICU-10201, 2240 for ICU-20201, and 1440 for ICU-30201. These numbers represent good starting points for the respective sensors, but you may find that your system requires additional timing margin, so starting with a number that is slightly smaller is a good idea. After setting the total length to the appropriate value in `measurement_config.c`, we can then set `MEASUREMENT_INTERVAL_MS` to 2 to get near 100% duty cycle sensor operation (because we want to listen for 2ms in this example).

4.3 EDITING THE MEASUREMENT CONFIGURATION

As mentioned in the previous section, the ICU-x0201 EVK software can be used to generate a measurement configuration C file similar to the one that ships with this example. This is the easiest way to edit the configuration for large changes. For minor changes, you can do some hand editing. This section describes this process.

4.3.1 Measurement configuration introduction

The measurement configuration is contained in a `measurement_queue_t` struct that directly maps to the ICU-x0201 memory. That is, the values you place in this struct are sent byte-for-byte to the ICU-x0201 data memory. The ICU-x0201 has a state machine that controls the measurement process; this state machine then reads this configuration directly from data memory and turns it into appropriate control signals for the PMUT. You can be sure that whatever configuration you place into the `measurement_queue_t` struct will be executed exactly by the ICU-x0201 (assuming you do not make SonicLib API calls which modify the configuration after importing it).

¹ These frequency values were pulled from the respective part datasheets on 8/22/2024. Please consult the latest datasheet for the most accurate numbers.

The measurement queue struct is called a *queue* because it contains two independent configurations that can be cycled through automatically by the ICU-x0201. That is, it is a two element, circular queue. There are additional global parameters that are not specific to one or the other measurement configuration. These are for configuring the interrupt and trigger pins and configuring the indices of the measurement queue that should be active.

The following sections detail the registers in the measurement queue configuration, including their offset addresses so they can be externally accessed. For details on how to read and write ICU registers, see Section 0.

4.3.2 Global parameters

intconfig

This register controls which pin of the ICU-x0201 is used as the data ready interrupt. It also has some alternate functions that allow for configuring an interrupt filter as well as a pulsed interrupt mode.

Bit 0 of this register controls which interrupt pin is used for data ready interrupts. Set bit 0 to 0 to select INT1 as the output or 1 to select INT2 as the output.

Bit 1 of this register is used to enable interrupt filtering. The purpose of interrupt filtering is to only have the ICU-x0201 generate interrupts when an on-chip algorithm generates an event rather than generating an interrupt for every measurement. Not all firmware variants support interrupt filtering. In GPT firmware, for example, enabling interrupt filtering will cause only interrupts with valid range values to be generated.

Bit 2 of this register is used to enable the pulsed interrupt mode. By default, the ICU-x0201 latches the data ready interrupt low and does not release it until the host has completed a SPI read. Some sensor operating modes require the pulsed mode to be used instead. For example, the high frame rate mode requires the use of the pulsed interrupt. This is to prevent a SPI read interrupt from being generated on the ASIC, which can interfere with the ongoing measurement due to the simultaneous measurement and readout.

meas_start

This register determines the starting index to use in the measurement queue. As stated above, the measurement queue contains two independent configs. These are enumerated as measurement configs 0 and 1. The ICU-x0201 has an additional state *current_meas* which is initialized to *meas_start*, then incremented every measurement up to *meas_stop*. After measurement *meas_stop*, *current_meas* is again set back to *meas_start*.

The measurement configuration which is run is not simply equal to *current_meas*. This is only true when *current_meas* ≤ 1. That is, config 0 is run when *current_meas* = 0 and config 1 is run when *current_meas* = 1. For values of *current_meas* beyond 1, config 1 is run.² This allows for setting up round-robin transmit sequences, described in the second example below.

For example, if *meas_start* = 0, *meas_stop* = 0, then only config 0 will be run. If *meas_start* = 0, *meas_stop* = 1, then the pattern will be 0, 1, 0, 1, and so on. If *meas_start* = 1, *meas_stop* = 1, then only config 1 is run.

As an example of a case where *meas_stop* exceeds 1, we can consider how to set up a round-robin transmit of multiple sensors³. If we have four total sensors and want them to take turns transmitting, what we should do is set config 0 to a transmit plus receive cycle and config 1 to a receive only cycle. Then we should set *meas_start* = 0, *meas_stop* = 3. Finally, we should set *current_meas* of each part to a different value in the range 0-3. The table below illustrates the detailed sequence of operations. The values of the inner cells are *current_meas*. The shaded cells indicate config 0 (TX + RX) is used. As can be seen from the table, the sensors will take turns transmitting.

² This behavior is configurable when using the init-ext firmware. See section 4.3.7.

³ This is how the high frame rate example is configured by default. No change to *meas_start*, *meas_stop*, or *current_meas* is necessary to enable round-robin mode.

Table 2. Round robin transmit example

time	Sensor 0	Sensor 1	Sensor 2	Sensor 3
t0	0	1	2	3
t1	1	2	3	0
t2	2	3	0	1
t3	3	0	1	2

meas_stop

See the description of *meas_start*.

current_meas

See the description of *meas_start*.

trigsrc

This register controls how measurements are triggered. The general options are hardware triggering by either pin, asynchronous triggering based on an internal low frequency clock, or continuous triggering. This register is one-hot encoded, and the bit descriptions are provided below.

If no bits are set, then no trigger mode is activated, though the sensor can still be triggered by a SPI command. It is sometimes useful to set this register to 0 when using a shared interrupt scheme to select which part will be triggered.

If bit 0 is set, the sensor is in hardware triggered mode and expects a trigger on the INT1 pin.

If bit 1 is set, the sensor is in hardware triggered mode and expects a trigger on the INT2 pin.

If bit 2 is set, the sensor is in self-timed (asynchronous) mode. Triggers are generated internally based on a low frequency clock. In this case, you need to set *meas_period*, which is a field that can be found in the measurement-specific parameters, documented below.

If bit 3 is set, the sensor is in continuous receive mode. Unlike the other modes, this is not a frame-based acquisition mode. Rather, the receiver samples the input continuously. Internally, this works by looping over the measurement configuration so that a measurement is never finished. Interrupts are generated when the receive operation for a given measurement config is done. Additionally, the receive buffer is split into two pieces so that data can be written and read at the same time. This means your receive measurement configuration must not generate more than the normal maximum number of samples divided by 2. This mode still respects the *meas_start* and *meas_stop* parameters, but it will not use different *odr* for the two (only the first is used).

reserved

This register is used for configuring double buffering and metadata packing options.

If bit 0 is set, the double buffer mode is enabled. This causes the on-chip receive buffer to be split into two parts so that one part can be written while the other is held for readout. The address of the buffer that should be read after a data ready measurement can be found by reading the *reserved2* field of the *raw_output_data* struct, or by using the SonicLib function *ch_get_next_buf_addr()*. For the buffer address to be updated correctly by SonicLib, either the *ch_interrupt()* or *ch_minimal_int_handler()* functions need to be called from the data ready interrupt handler of the host. Note that the effective maximum number of samples is reduced by a factor of 2 in the double buffer mode. You can find the maximum number of supported samples for a given firmware by checking the *ch_dev_t* struct *current_fw->max_samples* value and dividing by two (for double buffer mode).

If bit 1 is set, useful metadata will be copied into the first 4 bytes of the raw IQ data buffer, which are all normally 0. The first two bytes contain the address of the next buffer that should be read in little-endian format. The third byte indicates the measurement index which was used to produce the raw data⁴. The fourth byte is 1 if the buffer is valid and 0 otherwise. The SonicLib function `ch_update_metadata_from_iq0()` can be used to extract the metadata from the IQ data, update the values in the SonicLib device pointer struct, and set the first IQ sample back to 0. See the table below for a description of the first four IQ bytes read when bit 1 is set.

Table 3. IQ buffer when metadata option is enabled

Byte	Register name	Purpose
0	NEXT_BUF_ADDR[7:0]	LSByte of next buffer address
1	NEXT_BUF_ADDR[15:8]	MSByte of next buffer address
2	MEAS_IDX/(CUR_MEAS) ⁵	Measurement index used to generate data
3	BUF_VALID	1 if buffer is valid, else 0

The buffer for measurement n is valid from the time measurement n completes until the time measurement $n+2$ is triggered. See Figure 1. The metadata is written before the data ready interrupt is generated, meaning the metadata is also valid during the same period, but note the following:

When measurement $n+2$ is triggered, the meta data bytes are not overwritten atomically. Rather, the first two bytes will first be written to 0, then within approximately a few CPU clock cycles, the next two bytes are written to 0. Therefore, metadata with NEXT_BUF_ADDR equal to 0 indicates a read of an invalid buffer.

4.3.3 Measurement specific parameters

trx_inst

This array of `pmut_instruction` structs contains the instructions to be executed by the PMUT state machine. The array has a fixed length of 32 elements, meaning up to 32 instructions can be configured per measurement configuration.

Each `pmut_instruction` struct contains two registers: `cmd_config`, which selects and configures the type of instruction to run, and `length`, which sets the length of the instruction in clock cycles of the SMCLK (internal state machine clock). Both `cmd_config` and `length` are two byte registers stored in little-endian format.

The tables below document the function of all bits in the `pmut_instruction`. See AN-000175 *SonicLib Programmer's Guide* for a detailed description of the receiver and transmitter configuration bits.

⁴ In *icu-init-ext* firmware, this is instead the ASIC's internal `cur_meas` state, not the actual measurement configuration used. To find the measurement config that corresponds to the read value, you can use the `SEQ_TABLEx` registers. By default, measurement configuration 0 is run for `cur_meas=0` and measurement configuration 1 is run otherwise.

⁵ In *icu-init-ext* firmware, this is `CUR_MEAS`. See footnote above.

Table 4. PMUT Command Structure

	LSB					MSB
Byte 0	CMD[1:0]	RDY_IEN	DONE_IEN	RX_RESET	RX_BIAS_EN	CFG [9:8]=Byte0[7:6]
Byte 1	CFG[7:0]					
Byte 2	LENGTH[7:0]					
Byte 3	LENGTH[15:8]					

Table 5. PMUT Commands

CMD[1:0]	Description
00	Count
01	Transmit
10	Receive
11	End of file

Table 6: Count instruction format

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Byte 0	X	X	RX_BIAS_EN	RX_RESET	DONE_IEN	RDY_IEN	0	0
Byte 1	X	X	X	X	X	X	X	X
Byte 2	LENGTH[7:0]							
Byte 3	LENGTH[15:8]							

Table 7: TX instruction format

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Byte 0	X	X	RX_BIAS_EN	RX_RESET	DONE_IEN	RDY_IEN	0	1
Byte 1	Phase[3:0]				X	Pulse_Width[2:0]		
Byte 2	LENGTH[7:0]							
Byte 3	LENGTH[15:8]							

Table 8: RX instruction format

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Byte 0	0	0	RX_BIAS_EN=1	RX_RESET	DONE_IEN	RDY_IEN	1	0
Byte 1	X	RX_ATTEN[1:0]		RX_GAIN_REDUCTION[4:0]				
Byte 2	LENGTH[7:0]							
Byte 3	LENGTH[15:8]							

Table 9: EOF instruction format

Byte	Bit7	Bit6	Bit5	Bit4	Bit3	Bit2	Bit1	Bit0
Byte 0	X	X	X	X	DONE_IEN	RDY_IEN	1	1
Byte 1	X	X	X	X	X	X	X	X
Byte 2	0xXX							
Byte 3	0xXX							

Table 10. PMUT Command Register Descriptions

Register	Description
CMD	Transmit, receive, count, or end-of-file
RDY_IEN	Generate interrupt when peripheral ready for next data. Also disables DMA auto-load of next command. The ENABLE bit must be set again in software.
DONE_IEN	Generate interrupt when this command is done.
RX_RESET	Hold CIC filter in reset during this instruction.
RX_BIAS_EN	Enable the receiver bias for the duration of this instruction.
CFG	Command specific configuration (RX gain, TX phase, etc.)
LENGTH	Length of command in SMCLK cycles.

meas_period

This register is used to set the frame-rate in self-timed (asynchronous) sampling mode. The register is set in counts of the internal RTCCLK, which has a nominal frequency of about 29 kHz. The SonicLib function *ch_set_freerun_interval()* can be used to set this period in milliseconds, and SonicLib will correctly compute the value to set into the *meas_period* register, assuming the clock calibration step has been performed. The clock calibration is an optional step that can be done during sensor initialization that applies a fixed length pulse to an ICU interrupt pin in order to measure the frequency of RTCCLK. See the SonicLib Programmer's Guide for details.

The upper 2 bits of this register are used to control the time-hopping feature of the ICU parts. By default, time-hopping is enabled. This adds a random amount of jitter to the sample timing that can help with coexistence issues under certain circumstances. To disable time-hopping, the upper 2 bits of *meas_period* must be set to 1.

odr

This register contains the output data rate setting, which controls the intraframe data rate, or the number of samples recorded in one frame per unit time of receiving. The table below shows the impact of the *odr* setting. In the table, f_{op} is the frequency of the PMUT.

CIC_ODR	ODR	SMCLK Cycles
2	$f_{op}/32$	512
3	$f_{op}/16$	256
4	$f_{op}/8$	128
5	$f_{op}/4$	64
6	$f_{op}/2$	32

meas_flags

This register is used for additional measurement options.

If bit 0 is set, the data ready interrupt of the corresponding measurement configuration will always be suppressed. This can be useful in situations where you want one configuration to transmit-only and do not care about receiving the data.

4.3.4 Memory map

The tables below show the memory layout of the complete measurement queue structure. The addresses used are relative to a base address which must be read from address 0x1000 of the ICU part⁶ (DMEM access with offset 0; see Section 4.4). The base address can also be found from the SonicLib device pointer as the *sens_cfg_addr* member.

Table 11. Measurement queue memory map

Field	Relative Address ⁷	Fixed Address ⁸	Size / Type
intconfig	0x12	0x1026	uint8_t
meas_start	0x13	0x1027	uint8_t
meas_stop	0x14	0x1028	uint8_t
current_meas	0x15	0x1029	uint8_t
trigsrc	0x16	0x102A	uint8_t
reserved	0x17	0x102B	uint8_t
meas[0]	0x18	0x102C	measurement_t (132 bytes)
meas[1]	0x9c	0x10B0	measurement_t

Table 12. Individual measurement config map (meas[0] and meas[1])

Field	Relative Address	Size / Type
trx_inst[32]	0x00	32 element array of uint16_t cmd_config, uint16_t length
meas_period	0x80	uint16_t
odr	0x82	uint8_t
meas_flags	0x83	uint8_t

⁶ With icu_init_ext firmware version 1.1.0 or later (plugin version 2.2.0 or later), the base address is 0x1014 and is fixed.

⁷ Addresses are relative to the register map base address, which is always contained at DMEM physical address 0x1000.

⁸ Icu_init_ext firmware version 1.1.0 or later (plugin version 2.2.0 or later) ONLY. Not valid for other versions

4.3.5 Instruction extensions

The high frame rate example supports extended instruction sequences so that sequences longer than 32 instructions can be used. This is supported by the *icu_init_ext* firmware, which is the firmware distributed with the high frame rate example versions 3.4 and greater.

The *icu_init_ext* firmware has two new registers available at addresses 0x1008 and 0x100A, shown in Table 13. These define the addresses of the instruction extensions.

Usage of instruction extensions is documented extensively in the included document *icu_init_ext_api_doc_*.pdf*.

Table 13. Instruction extension registers

Register	Address	Size / Type
EXT0_ADDR	0x1008	uint16_t
EXT1_ADDR	0x100A	uint16_t

4.3.6 Additional configurations

The high frame rate example supports additional configurations beyond the original two available in most other firmware variants. These are not full configurations, as they do not support the *meas_period*, *odr*, or *meas_flags* fields of the *measurement_t* structure as defined by Table 12. This means that in general, these parameters are not supported when these additional configurations are used. That is, you cannot use free-running mode, cannot set different *odr* values per configuration⁹, or suppress the data ready interrupt when using the additional configurations.

The registers used to read and write the addresses of the additional configurations are show in Table 14. Refer to *icu_init_ext_api_doc_*.pdf* for detailed usage information.

Table 14. Additional measurement configuration registers

Register	Address	Size / Type
EXT2_ADDR	0x100C	uint16_t
EXT3_ADDR	0x100E	uint16_t

4.3.7 Custom measurement configuration sequencing

The high frame rate example supports custom sequencing of the four available measurement configurations. By default, configuration 0 (*meas[0]* with optional *EXT0*) is run when *cur_meas*=0, and configuration 1 (*meas[1]* with optional *EXT1*) is run otherwise. This behavior can be changed by programming a table that maps *cur_meas* to the desired configuration. The table is programmed into the registers defined in Table 15.

Each register defines 8 entries of the table. Each entry is a measurement configuration index for a particular value of *cur_meas*. Each measurement configuration index is 2-bits, allowing values of 0-3. A total of 16 measurement configuration indices can be programmed into the table. The number of indices programmed should be at least *meas_stop*+1 to ensure that every possible value of *cur_meas* has the desired corresponding measurement configuration index.

The individual 2-bit measurement configuration indices are documented in Table 16.

⁹ The odr of EXT2 and EXT3 is fixed to the ODR of meas[1]

Table 15. Custom sequence registers

Register	Address	Size / Type	Default
SEQ_TABLE0	0x1010	uint16_t	0x5554
SEQ_TABLE1	0x1012	uint16_t	0x5555

Table 16. cur_meas to config mapping

cur_meas	Register	Mask	Default
0	SEQ_TABLE0	0x0003	0
1	SEQ_TABLE0	0x000C	1
2	SEQ_TABLE0	0x0030	1
3	SEQ_TABLE0	0x00C0	1
4	SEQ_TABLE0	0x0300	1
5	SEQ_TABLE0	0x0C00	1
6	SEQ_TABLE0	0x3000	1
7	SEQ_TABLE0	0xC000	1
8	SEQ_TABLE1	0x0003	1
9	SEQ_TABLE1	0x000C	1
10	SEQ_TABLE1	0x0030	1
11	SEQ_TABLE1	0x00C0	1
12	SEQ_TABLE1	0x0300	1
13	SEQ_TABLE1	0x0C00	1
14	SEQ_TABLE1	0x3000	1
15	SEQ_TABLE1	0xC000	1

4.4 READING DATA

The example demonstrates how to read data using the recommended SonicLib APIs. This is the most straightforward way since it will keep the SonicLib structs up to date and extract metadata from the raw IQ data for you. There are a couple things to be aware of:

1. You must call the `ch_minimal_int_handler()` function from the data ready interrupt. This keeps some state in the SonicLib driver up to date.
2. You must call the `ch_update_metadata_from_iq0()` function after the readout is complete. This updates the SonicLib device pointer with the address of the next buffer that should be read.

If you do not wish to use SonicLib to assist with data readout, then you can do so manually. On each data ready interrupt, you need to read from the correct buffer address. The first buffer written will always be the one with the lower address. The address of the buffers is not the same across all firmware variants or firmware versions. To compute the buffer addresses, first read the base address from ICU register 0x1000 or reference the `sens_cfg_addr` device pointer member¹⁰. The offset of the first IQ data buffer is 0x1D0. To find the offset of the second buffer, you need to add `current_fw->max_samples*2` to the first buffer offset.

Alternatively, if the option to write metadata into the first IQ sample is enabled, the address of the next buffer to read can be found from the first 2 bytes of the IQ data. See documentation of the *reserved* register above.

Table 17. IQ data relative address

Field	Relative Address	Fixed Address ¹¹	Size / Type
IQdata	0x1d0	0x11e4	array of (int16_t, int16_t) IQ data pairs

¹⁰ In `icu_init_ext` firmware 1.1.0 or later (plugin version 2.2.0 or later), the base address is 0x1014 and is fixed.

¹¹ Valid only for `icu_init_ext` firmware 1.1.0 or later (plugin version 2.2.0 or later)

4.4.1 Timing Requirements

The buffer for measurement n is valid from the time measurement n completes until the time measurement $n+2$ is triggered. See Figure 1. The time a measurement *completes* should be interpreted as the time the data interrupt is generated.

There is no specific timing requirement between the measurement trigger and the start of a SPI read. In fact, the SPI read may be started *before* the next trigger, as is usual for single-buffer operation. However, it may be tricky from the host side to guarantee accurate trigger generation timing if the host is in the middle of a SPI read, which is why in our example we start the SPI read after the next trigger.

The measurement trigger starts an ISR on the ICU ASIC. This is a short ISR that preps and starts the measurement by writing into a control register of the PMUT state machine. This interrupt cannot be pre-empted and does not otherwise require memory access (though obviously the PMUT state machine does write into memory once the measurement is active). The point is that there is no conflict of resources between reading data over SPI and simultaneously triggering the sensor. Once the measurement is active, then hardware DMA arbitration ensures that the SPI read operation can occur simultaneously with PMUT state machine writing new measurement data as it becomes available.

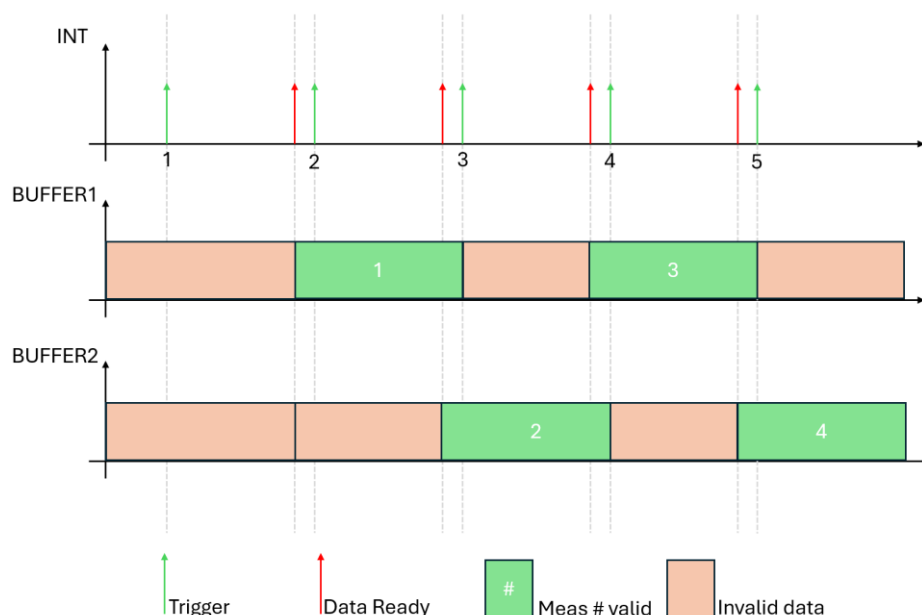


Figure 1. Data valid timing diagram

4.5 SPI DMA INTERFACE

The ICU ASIC contains a SPI peripheral device that can be addressed by a SPI controller on a host MCU or AP.

The SPI slave interface allows direct memory access to the ICU's entire program and data memory (PMEM/DMEM) address space. The registers described in Sections 4.3.2 and 4.3.3 are contained in the DMEM address space. For example, to write the *intconfig* register described above, you first read from 0x1000 (DMEM read with offset 0) to obtain the base address of the register map. Then add 0x12 to that address to obtain the address of *intconfig* register. Finally, perform a DMEM SPI write to that address offset to set the register.

Similarly, the raw IQ data is also contained in the DMEM address space. The IQ data buffer start address is found by adding 0x01D0 to the register map base address, as specified in Section 4.4.

4.5.1 Functional Description

Transaction Format:

- Individual transactions are delimited by the chip select pin, and deselecting the slave resets the transaction state machine. A transaction is a 2-byte command optionally followed by any number of data bytes.
- For DMA read transactions, the command must be followed by one don't-care byte to allow the first byte of data to be synchronized into the SPI clock domain.
- SPI data format: 8-bit word size and MSB-first. Clock idles high, data is output on the falling edge and captured on the rising edge.
- Commands and data use little-endian byte order.

4.5.2 DMA Access

Table 18. SPI DMA Command Format

MSB				LSB
[7]	[6]	[5]	[4:0]	[7:0]
0	1: Write 0: Read	1: PMEM 0: DMEM	Address [12:0]	

Shasta has two separate SRAMs dedicated to data and program memory, which are mapped to different parts of the physical address space. The command's address field is the offset of the first byte to be accessed from the start of the address space each SRAM is mapped to, as shown in the table below. The valid PMEM address range is 0xE800-0xFFFF. Writes to the invalid address range will be ignored, and the value of data read from these addresses is undefined.

Table 19. ICU PMEM and DMEM Access

SRAM instance (selected by the MEM bit)	Shasta address space
PMEM (6KB)	0xE000 + Address [12:0] (0xE000-0xE7FF do not map to physical memory)
DMEM (4KB)	0x1000 + Address [11:0]

DMA accesses auto-increment the address after every byte the master reads or writes, wrapping around to the start of the selected SRAM's address space. Note that DMA reads access memory one 16-bit word at a time and writes access memory one byte at time – the master cannot write a word atomically.

5 THEORY OF OPERATION

The high frame rate example uses the double-buffering mode of the ICU-x0201 to achieve very high sample rates, up to about 500 Hz. This section discusses the approach used and mentions some modifications that can be made.

In this example, we configure a timer to give us a tick every 1ms. This is the base unit of timing, and it is used to control the frame rate. Each time the system wakes up from the millisecond timer, it checks a flag to see if it should issue a new measurement trigger. In the event a measurement trigger is needed, it first pulses the interrupt line low, and then it starts a data-readout, which will be the data recorded from the previous frame.

The reason that the data readout is performed *after* the measurement trigger is to ensure accurate trigger timing that is not impacted by the time it takes to set up the non-blocking read of the previous measurement data. Other schemes are possible, such as wiring the interrupt pin directly to hardware control by the timer so that it is never subject to software delay.

This approach assumes that the measurement configuration has been adjusted properly so the ultrasonic measurement has time to complete before the next trigger will be issued. That is, you must set the total length of the config such that the worst-case total measurement time does not exceed the time in between triggers.

An alternate approach would be to allow the ICU part itself to control the frame rate. This can be done by immediately issuing a new trigger upon receiving a data ready interrupt from the sensor.

The screenshot below shows proper operation of the example when using two ICU-10201 sensors. In this case the total measurement length has been set to about 5000 and the measurement interval is set to 2ms. In the figure, you can see things begin when a data ready interrupt is received by the host. Shortly after, it issues a new measurement trigger, then starts the data readout. The data readout of the previous frame occurs during the new on-going measurement. Below, two sensors are read out, but the chip select is only shown for the first sensor. The timing margin in the system is essentially the time between the data-ready and trigger interrupts. If there is not enough margin, then either the measurement length must be reduced, or the measurement interval must be increased.



Figure 2. Logic analyzer capture of double-buffer mode

In the next figure, we have zoomed in on the very beginning of the data readout (note CS going low as the very beginning of the frame). Only a single SPI readout is necessary per interrupt per sensor. This is possible because we are using the option to embed metadata in the first 4 bytes of IQ data, which is controlled by a bit in the *reserved* field of the measurement queue configuration. In particular, we can see the first four bytes read are 0x28, 0x12, 0x01, and 0x01. This tells us that the next address we should read from (on the next data ready interrupt) is 0x1228 (little-endian format). It also tells us that the measurement data we are about to

read is from measurement config 1¹² (third byte) and that this is valid measurement data (fourth byte). If the fourth byte is ever 0 (or the first two bytes are 0), it indicates we have read from the wrong buffer, and the measurement data should be discarded.

In the below screenshot, you can also see the SPI read command that sets up the IQ data readout. The command bytes are 0x78, 0x07, followed by a dummy 0x00 byte to give the ICU time to prepare the data for read. The full 2-byte SPI command is then 0x0778 (little-endian format), which specifies a SPI DMEM read of offset 0x0778, which corresponds to physical address 0x1778. Note that the IQ data address is not constant across firmware versions, so these specific addresses are subject to change when firmware updates are provided. The IQ base address must be found by applying an offset to the register map base address, as specified in Section 4.4.

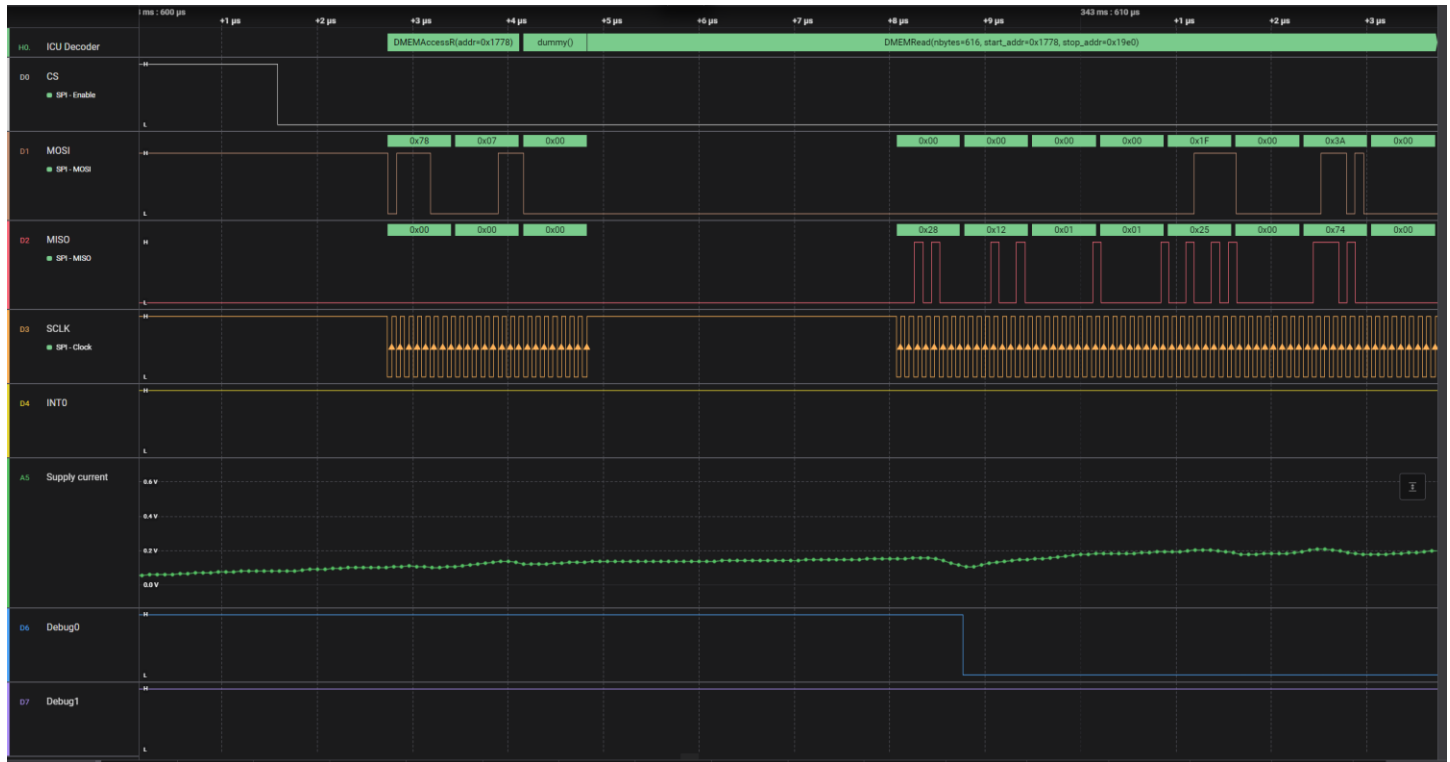


Figure 3. Metadata embedded in IQ data

¹² In *icu-init-ext* firmware, this is instead the ASIC's internal *cur_meas* state, not the actual measurement configuration used. To find the measurement config that corresponds to the read value, you can use the *SEQ_TABLEx* registers. By default, measurement configuration 0 is run for *cur_meas=0* and measurement configuration 1 is run otherwise.

6 REVISION HISTORY

Revision Date	Revision	Description
August 23, 2024	1.0	Initial Release
August 28, 2024	1.1	Add details on SPI DMA interface
February 19, 2025	1.2	Add timing diagram for double buffer readout
June 24, 2025	1.3	Add information on instruction extensions
June 27, 2025	1.4	Update extension register addresses based on change in icu-init-ext firmware. Add information on fixed register addresses.
July 14, 2025	1.5	- Add documentation of additional instruction extensions (now up to 4). - Add documentation of cur_meas to instruction sequence mapping table
July 18, 2025	1.5.1	Clarify that when using icu-init-ext firmware, the IQ metadata (if enabled) contains the 'cur_meas' value, not the actual meas config index.
August 11, 2025	1.5.2	Fix typos

This information furnished by InvenSense or its affiliates ("TDK InvenSense") is believed to be accurate and reliable. However, no responsibility is assumed by TDK InvenSense for its use, or for any infringements of patents or other rights of third parties that may result from its use. Specifications are subject to change without notice. TDK InvenSense reserves the right to make changes to this product, including its circuits and software, in order to improve its design and/or performance, without prior notice. TDK InvenSense makes no warranties, neither expressed nor implied, regarding the information and specifications contained in this document. TDK InvenSense assumes no responsibility for any claims or damages arising from information contained in this document, or from the use of products and services detailed therein. This includes, but is not limited to, claims or damages based on the infringement of patents, copyrights, mask work and/or other intellectual property rights.

Certain intellectual property owned by InvenSense and described in this document is patent protected. No license is granted by implication or otherwise under any patent or patent rights of InvenSense. This publication supersedes and replaces all information previously supplied. Trademarks that are registered trademarks are the property of their respective companies. TDK InvenSense sensors should not be used or sold in the development, storage, production or utilization of any conventional or mass-destructive weapons or for any other weapons or life threatening applications, as well as in any other life critical applications such as medical equipment, transportation, aerospace and nuclear instruments, undersea equipment, power plant equipment, disaster prevention and crime prevention equipment.

©2023 InvenSense. All rights reserved. InvenSense, MotionTracking, MotionProcessing, MotionProcessor, MotionFusion, MotionApps, DMP, AAR, and the InvenSense logo are trademarks of InvenSense, Inc. The TDK logo is a trademark of TDK Corporation. Other company and product names may be trademarks of the respective companies with which they are associated.